Transformer

A transformer learns which words matter to which other words, then uses that information to predict the next word correctly.

1️⃣ From raw sentence → tokens (learning step)

Input sentence

```
"I love AI"
```

Preprocessing

- Remove symbols
- Lowercase
- Tokenize

Result:

```
["i","love","ai"]
```

Tokens count = 3

Learning idea:

> Model never sees sentences, it only sees tokens.

2️⃣ Tokens → vectors (why this is needed)

Neural networks **cannot learn from words**, only numbers.

One-hot encoding (old way)

```
i    →[1,0,0]
love →[0,1,0]
ai   →[0,0,1]
```

Problems:

- No meaning
- No similarity
- Huge vectors

Embeddings (used in transformers)

Instead:

```
i    →[0.12, 0.77, -0.33, ...]
love →[0.91, -0.10, 0.44, ...]
ai   →[-0.22, 0.58, 0.81, ...]
```

Learning idea:

> Similar words → similar vectors

This is where **learning begins**.

3️⃣ Positional encoding (why order matters)

Sentence:

```
"I love AI"
```

vs

```
"AI love I"
```

Same words, different meaning.

So transformer **adds position info** to embeddings.

Learning idea:

> Word meaning + word position = correct understanding

4️⃣ Self-Attention (THIS is the core learning step)

Let's focus on one word:

Target word: "love"

Question the model learns to ask:

> "Which other words help me understand love?"

Attention behavior (conceptually)

```
love →i     (medium)
love → love  (ignore)
love → ai    (high)
```

So the model learns:

> "love" is strongly connected to "ai"

That relationship is **learned from data**, not hardcoded.

5️⃣ Why MULTI-HEAD attention helps learning

Instead of one attention view, transformer uses **multiple heads**.

Example:

- Head 1 → grammar
- Head 2 → meaning
- Head 3 → sentiment
- Head 4 → subject–object relation

Learning idea:

> Different heads learn different relationships at the same time

This is why representations become rich.

6️⃣ Add & Norm (why training doesn’t break)

Two problems during training:

- Values explode
- Learning becomes unstable

Add (residual connection)

- Keeps original information
- Prevents loss of signal

Norm (normalization)

- Mean = 0
- Std = 1

Learning idea:

> Stabilizes learning so deep networks don’t collapse

You saw:

```
z = (x - μ) / σ
```

That’s exactly what’s happening.

7️⃣ Feed Forward Network (where transformation happens)

After attention:

- Each token has **context**

- Now apply a small neural network

Learning idea:

> Attention decides what matters
>
> Feed-forward decides *how to transform it*

This happens **independently for each token**.

8️⃣ Encoder stack (why Nx blocks)

Why repeat encoder blocks?

Because:

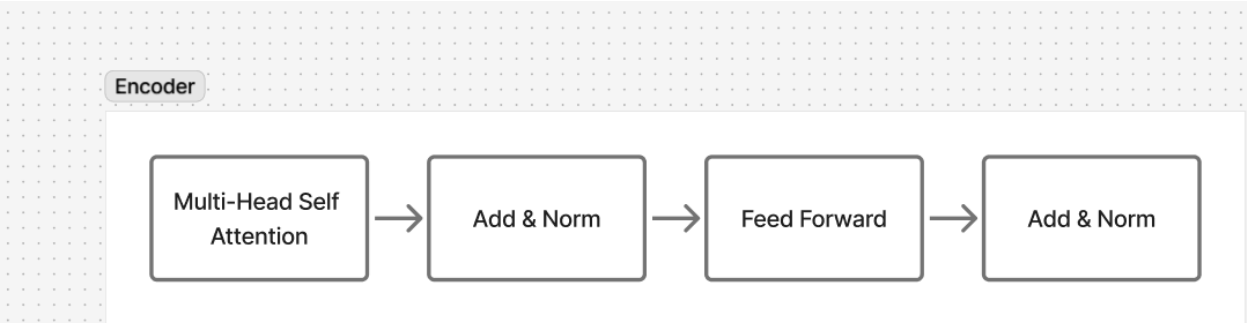
- First layer learns simple relations
- Deeper layers learn abstract meaning

Example progression:

- Layer 1 → word relations
- Layer 4 → phrase meaning
- Layer 8 → sentence intent

Learning idea:

> Depth = abstraction



9️⃣ Decoder (how generation is learned)

Decoder has **masked self-attention**.

Why masked?

Because when predicting next word:

> You must NOT see the future.

Example:

```
"I love ___"
```

Decoder can see:

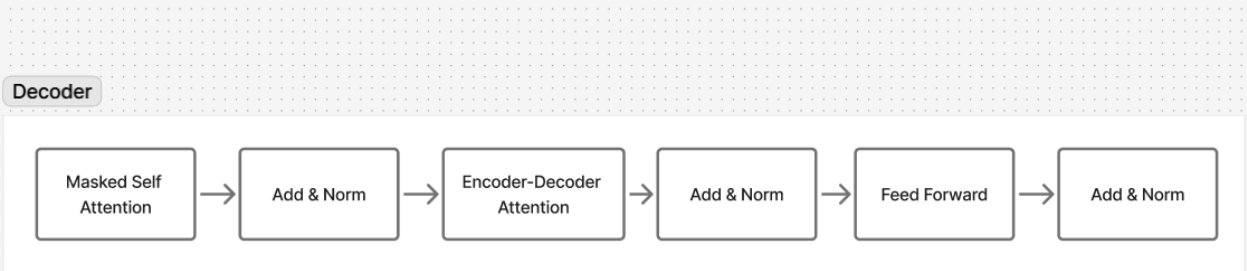
- "I"
- "love"

Decoder CANNOT see:

- future word

Learning idea:

> This forces the model to genuinely learn prediction.



🔟 Softmax → probability (final learning signal)

Final output:

```
word probabilities:
AI   →0.72
```

```
ML   →0.15
food →0.02
```

Correct word = **AI**

Loss is computed:

- If correct → reward
- If wrong → penalty

That's how **learning updates weights**.

11. Embedding Matrix (Core Representation Layer)

What the embedding matrix is

- Vocabulary size = number of unique tokens
- Embedding dimension = size of each word vector

The embedding matrix:

```
W_E ∈R^(V × D)
```

Where:

- V = vocabulary size (e.g. 50,257 tokens)
- D = embedding dimension (e.g. 12,288)

Each row corresponds to **one token**.

Example:

```
Token:"apple"
Index:1532
Embedding: W_E[1532] → [0.12, -0.88,1.03, ...]
```

Key point:

- One-hot vectors only **select rows**
- Learning happens inside **W_E**, not in one-hot

Why we don’t use average embeddings

If we average embeddings:

```
"I love AI"
avg = (vec(I) +vec(love) +vec(AI)) /3
```

Problem:

- Loses word order
- Loses relationships
- Same result for many sentences

So:

> We need contextual embeddings, not average embeddings

---

12. Contextual Embeddings (Why Self-Attention Exists)

Goal:

> Each token should share context with other tokens in the same sentence.

Example sentences:

- Apple is a fruit
- Apple is a company
- I have apple shares

Same word → different meaning

Same embedding → **not acceptable**

So:

- We pass embeddings through **self-attention**
- Output = **contextual embedding**

---

13. Self-Attention (Conceptual View)

For **each token**, we compute how much attention it should pay to **every other token** (including itself).

This answers:

> "Which words matter for understanding this word?"

Q, K, V (learning intuition)

For a focused word (example: `"Apple"`):

- **Query (Q)** → what I'm looking for
- **Key (K)** → what each word offers
- **Value (V)** → information to take if relevant

All three are **learned linear projections** of embeddings.

Attention score (conceptual)

For each pair:

```
score = similarity(Query, Key)
```

Then:

- Normalize scores (softmax)
- Use them to weight Values
- Sum → new embedding

Result:

> A new vector that represents “Apple in this context”

---

14. Why Multi-Head Attention

Single attention = single perspective.

Multi-head attention:

- Head 1 → syntactic relations
- Head 2 → semantic meaning
- Head 3 → long-range dependency
- Head 4 → entity relationships

Outputs are concatenated and projected.

Result:

> Richer contextual embedding

---

15. Why Add & Norm Is Mandatory

Add (Residual Connection)

- Prevents loss of original signal
- Helps gradient flow

Norm (Layer Normalization)

- Mean = 0
- Std = 1

Formula intuition:

```
z = (x − μ) / σ
```

Result:

- Stable training
- Deep stacks don’t explode

---

16. From Embeddings to Prediction

Pipeline:

```
Embedding
→PositionalEncoding
→Self-Attention
→Multi-HeadAttention
→Add&Norm
→FeedForward
→Add&Norm
→(repeatNxtimes)
→Decoder
→Softmax
→Nexttoken
```